

MSE 402: Computational Process Design



Term Project Report

**Writing our own code for periodic LJ system and
benchmark it against LAMMPS**

Adit Rambhia (21110009)

Nishant Tatar (21110223)

Vihaan Jhaveri (21110236)

Defining the system:

The system consists of a 500 single atom system of argon atoms simulated at a temperature of 1000 K using a microcanonical (NVE) ensemble. The interaction parameters for the LJ potential are defined for Argon, $\epsilon = 0.238$ and $\sigma = 3.405$.

The timestep for the simulation is set to 10 femtoseconds, with initial velocities assigned following a Gaussian distribution. A higher temperature gives a wider spread of the velocity distribution.

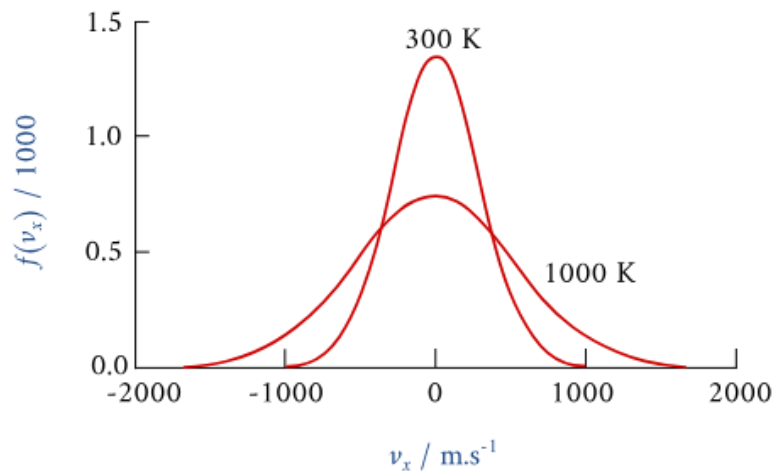


Fig.1: Gaussian distribution at different temperatures[1]

While a Maxwell Boltzmann distribution would be more exact for a system starting out, LAMMPS only has gaussian and uniform distributions for velocity, and as such the velocities are initialised using a gaussian distribution[2][3][4].

The simulation box follows periodic boundary conditions and has a size of 200 Å, which gives a density of 0.0041g/cc, about 2-3 times more than air.

Python Code:

The code is hosted here: https://github.com/Dashlander/MSE-402/blob/main/Project/MD2_0.py

Firstly, the necessary libraries are imported, namely scipy, numpy, and time libraries for python. To record the runtime, the start_time variable is used, which records the system time at the start. Then we define the simulation parameters, as well as constants required for computation. We precalculate and store these to reduce computational overhead. A random seed is set to ensure reproducibility.

The Initialisation function generates positions. We can use uniform or even normal distributions. I have used the normal distribution here to allow for some fluctuations in the potential energy calculations afterwards. Any atoms that go out of the simulation box by the spread are wrapped around into the box. The velocities are then generated. Both are $N \times 3$ arrays, which store values of x, y and z components of the positions and velocities vector. The LJ potential and force are calculated. To simplify calculations, a mask is created so that calculations only for atoms within the cutoff radius are done, to reduce computational work. The kinetic energy and the temperature are calculated. The berendsen thermostat is used, due to extreme fluctuations observed in the potential and kinetic energy due to numerical drift, and to bring these back to acceptable values, this has been implemented. A function to dump atom positions at given timesteps is created, to ensure the dump file is created in a format that can be read by OVITO.

The Main loop then calls the position and velocity vectors. An array of size $N \times N \times 3$ is created, where each value in the $N \times N$ grid corresponds to the distance between the i and j atoms, and the x 3 expansion stores the values for all three axes in x, y and z. The force is calculated, where the force is a $N \times N \times 3$ array as well. A new array of size $N \times 3$ is generated, which stores the total force vectors on the particular atom.

Then the simulation is started for the given number of time steps. The positions and velocities are updated according to the velocity verlet scheme. Forces are calculated at each new instant, and the velocities are calculated from the current and previous time step forces. The thermostat is applied every 10 iterations. Every 100 iterations, the kinetic energy, temperature, as well as the potential energy is calculated and logged, and the positions are stored to the dump file.

Once the simulation is complete, the start time is subtracted from the end time to find the time it took for the simulation to complete.

LAMMPS script:

The LAMMPS script: <https://github.com/Dashlander/MSE-402/blob/main/Project/Argon.in.txt>

This LAMMPS script is a molecular dynamics simulation of 500 argon atoms, employing the Lennard-Jones potential. The system is in real units; periodic boundary conditions were applied in a 3D simulation box that extends from 0 to 200 Å in each dimension. An Argon gas atom with a mass of 39.95 amu is defined, and 500 atoms were placed randomly inside the box. The atomic interactions are modeled by a Lennard-Jones potential: $\epsilon = 0.238$, $\sigma = 3.405$, and a cutoff distance of 10.0 Å.

Initial velocities are assigned through a Gaussian distribution to correspond to a temperature of 1000 K. The simulation is set to follow the NVE ensemble, meaning constant energy, volume, and particle count. Timestep is 10 fs. Thermodynamic properties of interest, namely kinetic energy, potential energy, total energy, temperature, and density are logged every 1000 steps. The script contains an energy minimization step, before running the dynamics for 100,000 steps. During the simulation time, the atomic positional coordinates get saved periodically to a dump file 'dump.lammpstrj.' The position and velocity vectors for each atom are then saved in a file at the end of the simulation.

Simulation results:

OVITO simulation of LAMMPS script:

<https://drive.google.com/file/d/1txuJaXvZVXbPBpIH-hQ6kPq2dfQtXI37/view?resourcekey>

OVITO simulation of Python script:

https://drive.google.com/file/d/1-yiQVzUqS5wUg6XWB_8MPGO-b54NLC3A/view?usp=drive_link

Benchmarking and Analysis of Results:

Runtimes:

- The simulation times are in an obvious contrast. LAMMPS, written in CPP, with its memory management as well other runtime optimisations, finished the simulation in 3.13 seconds, while Python took 3519.02 seconds for a simulation of a system with equivalent parameters. This is due to the fact that python is a high level language, and as such cannot achieve the same level of performance as a low level language such as CPP.

Potential Energy:

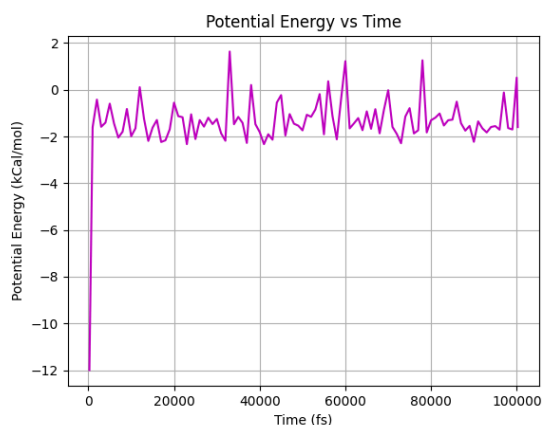


Fig.2: LAMMPS Potential Energy Plot

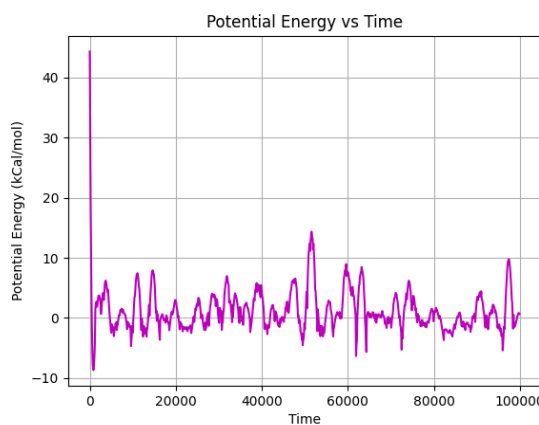


Fig3. Python Potential Energy Plot

As observed from the plots, while the potential energy has different starting values, they stabilise towards a mean after a few time steps. This is consistent with the fact that the simulation reaches towards equilibrium.

Due to the nature of the data generated, the standard deviation is not a very good measure of statistical inference for potential energy. Due to which, another measure must be used. Here, we use the Median Absolute Deviation, which is defined as the median of the absolute values of the difference of all measures of the dataset with their median. The Potential Energy for the LAMMPS simulation has a Median Absolute Deviation of 0.348, while the PE for the python simulation is 1.46.

Kinetic Energy:

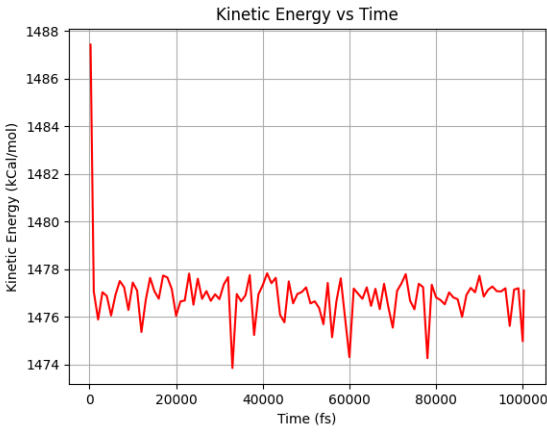


Fig.4: LAMMPS Kinetic Energy Plots

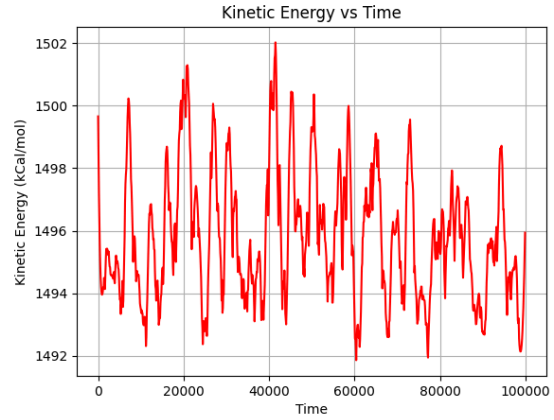


Fig.5: Python Kinetic Energy Plots

We can use the Coefficient of Variation(CoV), defined as the percentage of standard deviation from the mean calculated, for measuring the statistical deviance. The CoV here for the LAMMPS simulation is 0.087, while for the python run, it is 0.306.

Radial Distribution Function:

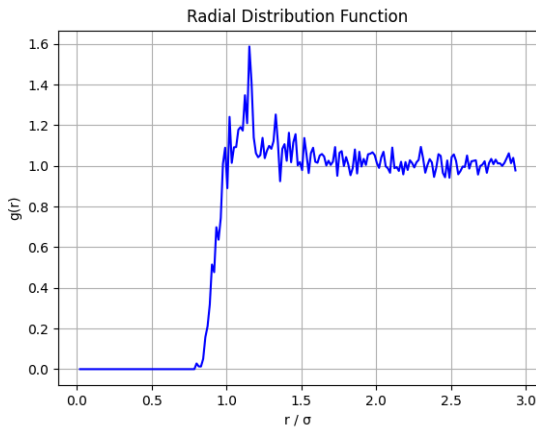


Fig.4: RDF Plot for the LAMMPS simulation

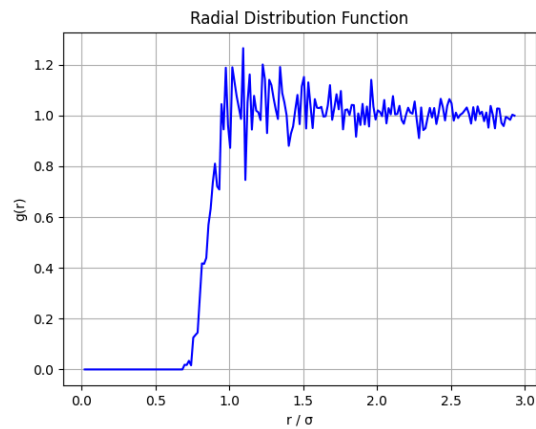


Fig.5: RDF Plot for the Python simulation

The RDF plots are calculated for the entire simulation run. An RDF plot is found for each time frame from the dump file, and then for the entire run, the RDF is averaged and plotted, as shown. Due to the fact that the simulation is at a higher temperature, there is not a sharp decay.

References:

[1] "27.2: The Gaussian Distribution of One Component of the Molecular Velocity," *Chemistry LibreTexts*, Jun. 21, 2014.

https://chem.libretexts.org/Bookshelves/Physical_and_Theoretical_Chemistry_Textbook_Maps/Physical_Chemistry%28LibreTexts%29/27%3A_The_Kinetic_Theory_of_Gases/27.02%3A_The_Gaussian_Distribution_of_One_Component_of_the_Molecular_Velocity

[2] "velocity command — LAMMPS documentation," *Lammps.org*, 2024. <https://docs.lammps.org/velocity.html>

[3] Materials Science Community Discourse, "Velocity Distribution," *Materials Science Community Discourse*, Feb. 12, 2015. <https://matsci.org/t/velocity-distribution/22822>

[4] *Maxwellian velocity distribution* (no date) *Maxwellian velocity distribution - AstroBaki*. Available at: https://casper.berkeley.edu/astrobaki/index.php/Maxwellian_velocity_distribution